

Absolutely — if you want **full jQuery study material from basics to important methods/functions, including syntax, examples, and explanations**, I'd structure it like a proper handbook rather than just a list of methods.

Complete jQuery Material – Basics to Important Methods

1. What is jQuery?

jQuery is a fast, lightweight JavaScript library designed to make common JavaScript tasks easier.

It simplifies:

- DOM manipulation
- Event handling
- Animations
- AJAX/API calls
- HTML/CSS manipulation
- Form handling
- Traversing HTML elements

JavaScript vs jQuery

JavaScript:

```
document.getElementById("title").style.color = "red";
```

jQuery:

```
$("#title").css("color", "red");
```

So jQuery basically provides **shorter and easier syntax for many common JavaScript operations**.

2. How to Include jQuery

There are two main ways.

Method 1: CDN

Add this inside your HTML `<head>` or before your own JavaScript:

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

Then:

```
<script>
  $(document).ready(function () {
    console.log("jQuery is ready!");
  });
</script>
```

Important

Your own jQuery code should generally come **after the jQuery library has been loaded**.

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="script.js"></script>
```

3. Basic jQuery Syntax

The basic syntax is:

```
$(selector).action();
```

For example:

```
$("#title").hide();
```

Here:

Part	Meaning
------	---------

Part	Meaning
<code>\$</code>	jQuery function
<code>"#title"</code>	Selector
<code>.hide()</code>	jQuery method/action

Another example:

```
$ ("p").hide ();
```

This hides all `<p>` elements.

4. The `$` Symbol

`$` is an alias for `jQuery`.

These are equivalent:

```
$ ("#box");
```

and:

```
jQuery("#box");
```

Usually developers use `$` because it is shorter.

5. Document Ready

A very important concept.

```
$(document).ready(function () {  
    // jQuery code  
});
```

Short version:

```
$(function () {  
    // jQuery code  
});
```

It means:

Execute this jQuery code after the HTML document has finished loading.

Example:

```
$(document).ready(function () {  
    $("#btn").click(function () {  
        alert("Button clicked!");  
    });  
});
```

6. jQuery Selectors

Selectors are used to find HTML elements.

Element Selector

```
$("p")
```

Selects all `<p>` elements.

```
$("h1")
```

Selects all `<h1>` elements.

ID Selector

HTML:

```
<h1 id="title">Hello</h1>
```

jQuery:

```
$("#title")
```

Class Selector

HTML:

```
<p class="message">Hello</p>
```

jQuery:

```
$(".message")
```

Universal Selector

```
$("*")
```

Selects all elements.

Multiple Selectors

```
$("h1, p, div")
```

Selects all `h1`, `p`, and `div` elements.

Attribute Selector

```
$("input[type='text']")
```

Selects text input fields.

```
$("a[target='_blank']")
```

Selects links having `target="_blank"`.

7. jQuery Effects

jQuery provides several built-in effects.

hide()

Syntax

```
$(selector).hide();
```

Example:

```
$("#box").hide();
```

With speed:

```
$("#box").hide(1000);
```

show()

```
$("#box").show();
```

toggle()

```
$("#box").toggle();
```

If the element is visible → hides it.

If hidden → shows it.

fadeIn()

```
$("#box").fadeIn();
```

With speed:

```
$("#box").fadeIn(1000);
```

fadeOut()

```
$("#box").fadeOut();
```

fadeToggle()

```
$("#box").fadeToggle();
```

fadeTo()

Used to fade an element to a particular opacity.

```
$("#box").fadeTo(1000, 0.5);
```

Here:

```
1000 → duration in milliseconds  
0.5  → opacity
```

8. Sliding Effects

slideDown()

```
$("#box").slideDown();
```

slideUp()

```
$("#box").slideUp();
```

slideToggle()

```
$("#box").slideToggle();
```

Example:

```
$("#btn").click(function () {  
    $("#menu").slideToggle();  
});
```

9. Callback Functions

Many jQuery methods can execute a function after the operation is complete.

Example:

```
$("#box").hide(1000, function () {  
    alert("Box has been hidden!");  
});
```

The function:

```
function () {  
    alert("Box has been hidden!");  
}
```


is called a **callback function**.

10. jQuery HTML / DOM Manipulation

These methods are extremely important.

text()

Gets or changes text.

Get

```
let value = $("#title").text();
```

Set

```
$("#title").text("Hello World");
```

html()

Gets or changes HTML.

Get

```
let value = $("#box").html();
```

Set

```
$("#box").html("<b>Hello</b>");
```

Difference

```
$("#box").text("<b>Hello</b>");
```

Displays:

```
<b>Hello</b>
```

while:

```
$("#box").html("<b>Hello</b>");
```

displays:

Hello

11. val()

Used mainly with forms.

HTML:

```
<input id="name" type="text">
```

Get value:

```
let name = $("#name").val();
```

Set value:

```
$("#name").val("Krisha");
```

12. attr()

Used to get/set HTML attributes.

HTML:

```

```

Get:

```
let image = $("#photo").attr("src");
```

Set:

```
$("#photo").attr("src", "new.jpg");
```

Multiple attributes:

```
$("#photo").attr({  
  src: "new.jpg",  
  alt: "New Image"  
});
```

13. removeAttr()

Removes an attribute.

```
$("#box").removeAttr("class");
```

14. prop()

Used mainly for DOM properties such as:

- checked
- disabled
- selected

Example:

```
$("#check").prop("checked", true);
```

Disable button:

```
$("#btn").prop("disabled", true);
```

Enable:

```
$("#btn").prop("disabled", false);
```

attr() vs prop()

For:

```
<input type="checkbox" id="check">
```

Use:

```
$("#check").prop("checked");
```

to check whether it is currently checked.

15. CSS Manipulation

css()

Get CSS

```
let color = $("#box").css("color");
```

Set one property

```
$("#box").css("color", "red");
```

Set multiple properties

```
$("#box").css({  
    color: "red",  
    backgroundColor: "yellow",  
    fontSize: "20px"  
});
```

16. addClass()

```
$("#box").addClass("active");
```

17. removeClass()

```
$("#box").removeClass("active");
```

18. toggleClass()

```
$("#box").toggleClass("active");
```

19.hasClass()

Checks whether an element has a class.

```
if ($("#box").hasClass("active")) {  
    console.log("Class exists");  
}
```

Returns:

```
true
```

or:

```
false
```

20. Adding HTML Elements

append()

Adds content at the end.

```
$("#list").append("<li>Apple</li>");
```

prepend()

Adds content at the beginning.

```
$("#list").prepend("<li>Apple</li>");
```

before()

Adds content before the selected element.

```
$("#box").before("<p>Before</p>");
```

after()

Adds content after the selected element.

```
$("#box").after("<p>After</p>");
```

21. Removing Elements

remove()

Removes the selected element completely.

```
$("#box").remove();
```

empty()

Removes all child elements/content.

```
$("#box").empty();
```

Difference:

```
$("#box").remove();
```

removes `#box` itself.

Whereas:

```
$("#box").empty();
```

keeps `#box` but removes everything inside it.

22. clone()

Creates a copy of an element.

```
let copy = $("#box").clone();
```

Then:

```
$("#container").append(copy);
```

23. Event Handling

Events are one of the most important parts of jQuery.

click()

```
$("#btn").click(function () {  
    alert("Button clicked");  
});
```

dblclick()

```
$("#box").dblclick(function () {  
    alert("Double clicked");  
});
```

mouseenter()

```
$("#box").mouseenter(function () {  
    console.log("Mouse entered");  
});
```

mouseleave()

```
$("#box").mouseleave(function () {  
    console.log("Mouse left");  
});
```

hover()

```
$("#box").hover(
  function () {
    console.log("Mouse entered");
  },
  function () {
    console.log("Mouse left");
  }
);
```

24. Keyboard Events

keypress()

```
$("#name").keypress(function () {
  console.log("Key pressed");
});
```

Note: In modern applications, `keydown` and `keyup` are generally preferred.

keydown()

```
$("#name").keydown(function () {
  console.log("Key down");
});
```

keyup()

```
$("#name").keyup(function () {
  console.log("Key released");
});
```

25. Form Events

submit()

```
$("#myForm").submit(function (event) {  
    event.preventDefault();  
  
    console.log("Form submitted");  
});
```

`preventDefault()` prevents the browser's default form submission.

change()

```
$("#country").change(function () {  
    console.log("Country changed");  
});
```

focus()

```
$("#name").focus(function () {  
    console.log("Input focused");  
});
```

blur()

```
$("#name").blur(function () {  
    console.log("Input lost focus");  
});
```

26. Modern Event Method – on()

Instead of writing:

```
$("#btn").click(function () {  
    alert("Hello");  
});
```

you can write:

```
$("#btn").on("click", function () {  
    alert("Hello");  
});
```

Syntax

```
$(selector).on(event, function);
```

Example:

```
$("#box").on("mouseenter", function () {  
    $(this).css("background", "yellow");  
});
```

27. off()

Removes an event handler.

```
$("#btn").off("click");
```

28. Event Object

```
$("#btn").on("click", function (event) {  
    console.log(event);  
});
```

Useful properties include:

```
event.target  
event.type  
event.currentTarget
```

29. `this` in jQuery

Inside an event handler:

```
$("p").click(function () {  
    $(this).hide();  
});
```

`this` refers to the particular `<p>` that was clicked.

This is extremely useful when working with multiple elements.

30. Traversing Elements

jQuery provides powerful traversal methods.

`parent()`

```
$("#child").parent();
```

Gets the immediate parent.

parents()

```
$("#child").parents();
```

Gets ancestors.

children()

```
$("#box").children();
```

Gets direct children.

find()

```
$("#box").find("p");
```

Finds `<p>` elements inside `#box`.

siblings()

```
$("#item").siblings();
```

Gets sibling elements.

next()

```
$("#item").next();
```

Gets next sibling.

prev()

```
$("#item").prev();
```

Gets previous sibling.

first()

```
$("p").first();
```

Selects the first matching element.

last()

```
$("p").last();
```

Selects the last matching element.

eq()

```
$("p").eq(2);
```

Selects the element at index **2**.

Remember:

```
Index starts from 0
```

31. filter()

```
$( "p" ).filter( ".active" );
```

Selects only matching elements.

32. not()

```
$( "p" ).not( ".active" );
```

Selects `<p>` elements that do NOT have the `active` class.

33. each()

Very important for loops.

```
$( "p" ).each( function () {  
    console.log( $( this ).text() );  
});
```

You can access the index:

```
$( "p" ).each( function ( index ) {  
    console.log( index );  
    console.log( $( this ).text() );  
});
```

34. Dimensions

width()

```
let width = $( "#box" ).width();
```

Set:

```
$("#box").width(300);
```

height()

```
let height = $("#box").height();
```

Set:

```
$("#box").height(200);
```

innerWidth()

```
$("#box").innerWidth();
```

Includes padding.

innerHeight()

```
$("#box").innerHeight();
```

outerWidth()

```
$("#box").outerWidth();
```

outerHeight()

```
$("#box").outerHeight();
```

35. Position

offset()

Gets the position relative to the document.

```
let position = $("#box").offset();

console.log(position.top);
console.log(position.left);
```

position()

Gets position relative to the offset parent.

```
let position = $("#box").position();
```

36. Scrolling

scrollTop()

Get:

```
let value = $(window).scrollTop();
```

Set:

```
$(window).scrollTop(0);
```

scrollLeft()

```
let value = $(window).scrollLeft();
```

37. Animation

jQuery has a general-purpose:

```
animate()
```

Syntax

```
$(selector).animate(properties, duration);
```

Example:

```
$("#box").animate({  
  left: "200px",  
  opacity: 0.5  
}, 1000);
```

Animation with callback

```
$("#box").animate({  
  width: "300px"  
}, 1000, function () {  
  console.log("Animation complete");  
});
```

38. stop()

Stops an animation.

```
$("#box").stop();
```

39. AJAX

AJAX allows a webpage to communicate with a server **without reloading the entire page**.

One of the important jQuery methods is:

```
$.ajax()
```

Basic syntax

```
$.ajax({  
  url: "https://example.com/api/data",  
  method: "GET",  
  success: function (data) {  
    console.log(data);  
  },  
  error: function (error) {  
    console.log(error);  
  }  
});
```

40. \$.get()

Used for GET requests.

```
$.get("https://example.com/api/users", function (data) {  
  console.log(data);  
});
```

41. \$.post()

Used for POST requests.

```
$.post(  
    "https://example.com/api/users",  
    {  
        name: "Krisha",  
        age: 22  
    },  
    function (data) {  
        console.log(data);  
    }  
);
```

42. \$.getJSON()

Used to retrieve JSON data.

```
$.getJSON("data.json", function (data) {  
    console.log(data);  
});
```

43. jQuery AJAX – Important Options

```
$.ajax({  
    url: "/api/users",  
    method: "POST",  
    contentType: "application/json",  
    data: JSON.stringify({  
        name: "John"  
    }),  
  
    success: function (response) {  
        console.log(response);  
    },  
  
    error: function (xhr) {  
        console.log(xhr);  
    },  
});
```

```
    complete: function () {  
        console.log("Request completed");  
    }  
});
```

Important options:

Option	Purpose
url	API/server URL
method	GET, POST, PUT, DELETE etc.
data	Data sent to server
success	Runs when request succeeds
error	Runs when request fails
complete	Runs after request finishes
contentType	Type of data being sent

44. jQuery Utility Functions

Some jQuery functions are used through `$` directly rather than on a selected element.

Example:

```
$.each()
```

`$.each()`

```
$.each(array, function (index, value) {  
    console.log(index, value);  
});
```

Example:

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
$.each(fruits, function (index, fruit) {  
    console.log(index, fruit);  
});
```

45. \$.trim()

Historically used to remove whitespace:

```
let value = $.trim("  Hello  ");
```

Modern JavaScript generally uses:

```
value.trim();
```

46. \$.isArray()

Older jQuery code may use:

```
$.isArray(value);
```

Modern JavaScript:

```
Array.isArray(value);
```

47. Data Storage

data()

Stores or retrieves data associated with an element.

Set:

```
$("#box").data("id", 101);
```

Get:

```
let id = $("#box").data("id");
```

Example:

```
$("#student").data("name", "Krisha");
```

```
console.log($("#student").data("name"));
```

48. Removing Data

```
$("#box").removeData("id");
```

49. Forms

Suppose:

```
<form id="myForm">
  <input type="text" id="name">
  <input type="email" id="email">
  <button type="submit">Submit</button>
</form>
```

jQuery:

```
$("#myForm").submit(function (event) {
  event.preventDefault();

  let name = $("#name").val();
  let email = $("#email").val();

  console.log(name);
```

```
console.log(email);  
});
```

50. Select and Checkbox

Checkbox

```
if ($("#terms").prop("checked")) {  
    console.log("Checked");  
}
```

Set checked:

```
$("#terms").prop("checked", true);
```

Uncheck:

```
$("#terms").prop("checked", false);
```

51. Method Chaining

One of jQuery's famous features is **method chaining**.

Instead of:

```
$("#box").hide();  
  
$("#box").css("color", "red");  
  
$("#box").fadeIn();
```

you can write:

```
$("#box")  
    .hide()
```



```
.css("color", "red")
.fadeIn();
```

This is called **method chaining**.

52. Selector + Method Combination

Example:

```
$("#p")
  .css("color", "blue")
  .slideUp(1000)
  .slideDown(1000);
```

This demonstrates:

- selector
 - CSS manipulation
 - animation
 - method chaining
-

53. Common jQuery Methods – Quick Revision

Category	Methods
Effects	<code>hide()</code>
Effects	<code>show()</code>
Effects	<code>toggle()</code>
Effects	<code>fadeIn()</code>
Effects	<code>fadeOut()</code>
Effects	<code>fadeToggle()</code>
Effects	<code>fadeTo()</code>
Effects	<code>slideUp()</code>
Effects	<code>slideDown()</code>

Category	Methods
Effects	<code>slideToggle()</code>
Animation	<code>animate()</code>
Animation	<code>stop()</code>
HTML	<code>text()</code>
HTML	<code>html()</code>
Forms	<code>val()</code>
Attributes	<code>attr()</code>
Attributes	<code>removeAttr()</code>
Properties	<code>prop()</code>
CSS	<code>css()</code>
Classes	<code>addClass()</code>
Classes	<code>removeClass()</code>
Classes	<code>toggleClass()</code>
Classes	<code>hasClass()</code>
DOM	<code>append()</code>
DOM	<code>prepend()</code>
DOM	<code>before()</code>
DOM	<code>after()</code>
DOM	<code>remove()</code>
DOM	<code>empty()</code>
DOM	<code>clone()</code>
Traversing	<code>parent()</code>
Traversing	<code>parents()</code>
Traversing	<code>children()</code>
Traversing	<code>find()</code>
Traversing	<code>siblings()</code>
Traversing	<code>next()</code>
Traversing	<code>prev()</code>

Category	Methods
Traversing	<code>first()</code>
Traversing	<code>last()</code>
Traversing	<code>eq()</code>
Traversing	<code>filter()</code>
Traversing	<code>not()</code>
Traversing	<code>each()</code>
Events	<code>click()</code>
Events	<code>dblclick()</code>
Events	<code>mouseenter()</code>
Events	<code>mouseleave()</code>
Events	<code>hover()</code>
Events	<code>keydown()</code>
Events	<code>keyup()</code>
Events	<code>change()</code>
Events	<code>focus()</code>
Events	<code>blur()</code>
Events	<code>submit()</code>
Events	<code>on()</code>
Events	<code>off()</code>
Dimensions	<code>width()</code>
Dimensions	<code>height()</code>
Dimensions	<code>innerWidth()</code>
Dimensions	<code>innerHeight()</code>
Dimensions	<code>outerWidth()</code>
Dimensions	<code>outerHeight()</code>
Position	<code>offset()</code>
Position	<code>position()</code>
Scroll	<code>scrollTop()</code>

Category	Methods
Scroll	<code>scrollLeft()</code>
AJAX	<code>\$.ajax()</code>
AJAX	<code>\$.get()</code>
AJAX	<code>\$.post()</code>
AJAX	<code>\$.getJSON()</code>
Utility	<code>\$.each()</code>
Data	<code>data()</code>
Data	<code>removeData()</code>

54. Important jQuery Interview/Viva Questions

What is jQuery?

A JavaScript library used to simplify DOM manipulation, events, animations, AJAX and other common tasks.

What is `$` ?

`$` is an alias for the `jQuery` function.

```
$("#id");
```

is equivalent to:

```
jQuery("#id");
```

What is a selector?

A selector identifies HTML elements.

```
$("#id")  
$(".class")  
$("p")
```

What does `$(document).ready()` do?

It executes code after the HTML document's DOM has been loaded.

Difference between `text()` and `html()` ?

`text()` works with plain text.

`html()` works with HTML content.

Difference between `attr()` and `prop()` ?

`attr()` deals with HTML attributes, while `prop()` is generally used for current DOM properties such as `checked`, `selected`, and `disabled`.

Difference between `remove()` and `empty()` ?

```
remove()
```

removes the element itself.

```
empty()
```

removes its contents while keeping the element.

What is method chaining?

Calling multiple jQuery methods on the same object:

```
$("#box").addClass("active").show().fadeIn();
```

What is AJAX?

★ The most important syntax to memorize

If you're teaching/studying jQuery, these are the core patterns I'd memorize first:

```
$(selector).method();
```

```
$(selector).method(value);
```

```
$(selector).method(function () {  
    // code  
});
```

```
$(selector).on("event", function () {  
    // code  
});
```

```
$(selector).css("property", "value");
```

```
$(selector).attr("attribute", "value");
```

```
$(selector).prop("property", value);
```

```
$(selector).each(function (index) {  
    // code  
});
```

```
$.ajax({  
    url: "...",  
    method: "GET",  
    success: function (data) {  
        // code  
    },  
    error: function (error) {  
        // code  
    }  
});
```

```
}  
}) ;
```

This gives you the **foundation for practically all the jQuery methods above**.